Hands-On Web Application Security Laboratory

Lab 1: Virtualization Environment Setup

Lab 2: Virtual Network Configuration

Lab 3: Linux Server Administration Basics

Lab 4: LAMP Stack Deployment

Lab 5: DVWA Vulnerable Web Application Deployment

Lab 6: HTTP Traffic Analysis

Lab 7: OWASP ZAP Vulnerability Scanning

Lab 8: Burp Suite Web Application Attacks

Lab 9: Web Application Firewall Implementation

LAB 1

1. Lab Title: Installing the Virtualization Environment for Web Application Security Labs

2. Introduction

Modern cybersecurity training requires safe environments where students can perform security experiments without affecting real systems. Virtualization technology allows multiple operating systems to run on a single physical computer while remaining isolated from the host system.

In cybersecurity laboratories, virtualization platforms are used to simulate networks, servers, and attacker machines. This enables students to deploy vulnerable applications, perform penetration testing, and analyze security defenses in a controlled environment.

In this lab, students will install a virtualization platform and create virtual machines that will be used throughout the Web Application Security Lab Series. The lab environment will consist of two main systems:

- **Kali Linux** – used as the attacker and security testing machine.
- **Ubuntu Server** – used as the target server hosting web applications.

These virtual machines will later be used to perform vulnerability scanning, penetration testing, and web application security experiments.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the concept of virtualization in cybersecurity environments.
- Install and configure VirtualBox virtualization software.
- Import a Kali Linux virtual machine.
- Create an Ubuntu Server virtual machine.
- Configure virtual machine resources such as RAM, CPU, and disk space.
- Prepare the lab environment for future cybersecurity experiments.

4. Significance of the Lab in Cybersecurity Training

Virtual laboratory environments are essential for cybersecurity education. They allow students to safely practice offensive and defensive security techniques without impacting real systems or networks.

Through virtualization, cybersecurity professionals can simulate real-world environments including servers, attackers, and network infrastructures. This allows students to develop practical skills in areas such as penetration testing, vulnerability assessment, and system security.

This lab serves as the foundation for all subsequent labs in the Web Application Security Lab Series, where students will deploy vulnerable web applications, perform attacks, and implement defensive security mechanisms.

5. Lab Scenario

A university cybersecurity course requires students to perform web application security testing in a controlled environment. To achieve this, a virtual lab environment must be created using virtualization technology.

Students will install a virtualization platform and create two virtual machines:

- A Kali Linux machine that will be used as the attacker system.
- An Ubuntu Server machine that will host vulnerable web applications.

These machines will later be used in labs involving vulnerability scanning, web attacks, and Web Application Firewall implementation.

6. Lab Environment Architecture

The lab environment consists of a host computer running VirtualBox with two virtual machines.

Order	Component	Function
1	Host Computer	Base physical system
2	VirtualBox	Hosts virtual machines
3	Kali Linux VM	Performs security testing
4	Ubuntu Server VM	Hosts web server services

7. Lab Requirements

Hardware Requirements

Minimum recommended system configuration:

Resource	Requirement
RAM	8 GB
Storage	40 GB free
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Security testing machine
Ubuntu Server	Target web server
Web Browser	Download software and images

8. Step 1: Download VirtualBox

1. Open a web browser.
2. Navigate to the VirtualBox website:

```
https://www.virtualbox.org
```

3. Click Downloads.
4. Download the appropriate version for your operating system:

- Windows hosts
- macOS hosts
- Linux distributions

9. Step 2: Install VirtualBox

1. Locate the downloaded VirtualBox installer.
2. Run the installer.
3. Follow the installation wizard.

Recommended installation settings:

- Enable USB support
- Enable networking features
- Install VirtualBox networking drivers

4. Click Finish to complete installation.

After installation, launch Oracle VM VirtualBox Manager.

10. Step 3: Download Kali Linux Virtual Machine

Navigate to the Kali Linux website:

```
https://www.kali.org/get-kali/
```

Download the Kali Linux VirtualBox image.

Example file:

```
kali-linux-virtualbox-amd64.ova
```

11. Step 4: Import Kali Linux Virtual Machine

Open VirtualBox Manager.

1. Click File
2. Select Import Appliance
3. Browse and select the Kali Linux .ova file.
4. Click Next
5. Review the virtual machine settings.
6. Click Import

VirtualBox will automatically create the Kali Linux virtual machine.

12. Step 5: Configure Kali Linux Virtual Machine Resources

After importing Kali Linux:

1. Select the Kali Linux VM.
2. Click Settings.

Recommended configuration:

System Settings

Parameter	Value
RAM	2048 MB
CPU	2 processors

Display Settings

Video Memory:

128 MB

Storage

Ensure the Kali Linux virtual disk is attached.

13. Step 6: Start Kali Linux Virtual Machine

1. Select the Kali Linux VM.
2. Click Start.

Login credentials (default):

Username: kali
Password: kali

Once logged in, verify the system is working correctly.

14. Step 7: Download Ubuntu Server ISO

Navigate to the Ubuntu website:

```
https://ubuntu.com/download/server
```

Download the latest Ubuntu Server ISO file.

Example file:

```
ubuntu-22.04-live-server-amd64.iso
```

15. Step 8: Create Ubuntu Server Virtual Machine

Open VirtualBox Manager.

Click:

```
New
```

Configure the VM:

Setting	Value
Name	Ubuntu-Server
Type	Linux
Version	Ubuntu (64-bit)

Memory Configuration

Recommended RAM:

```
4096 MB
```

CPU Configuration

Processors:

```
2 CPUs
```

Disk Configuration

Create a virtual hard disk.

Recommended size:

```
25 GB
```

Disk type:

VDI (VirtualBox Disk Image)

Allocation type:

Dynamically Allocated

16. Step 9: Attach Ubuntu Server ISO

Select the Ubuntu VM.

Open:

Settings → Storage

Under Controller IDE:

Click Empty Disk

Select:

Choose a Disk File

Attach the downloaded Ubuntu ISO file.

17. Step 10: Install Ubuntu Server

Start the virtual machine.

Follow installation steps:

1. Select Install Ubuntu Server
2. Configure language and keyboard
3. Configure user account
4. Configure storage
5. Install OpenSSH server (optional)

Once installation completes, reboot the VM.

18. Expected Results

Students should observe:

Result	Description
VirtualBox installed	Virtualization platform operational
Kali Linux VM running	Security testing machine available
Ubuntu Server VM created	Target server ready
Both systems operational	Lab environment prepared

19. Deliverables

Students must submit the following screenshots:

1. VirtualBox Manager showing installed virtual machines
2. Kali Linux desktop running
3. Ubuntu Server login screen
4. VM configuration showing RAM and CPU settings

20. Discussion Questions

1. What is virtualization and why is it important in cybersecurity labs?
2. What advantages do virtual machines provide over physical machines?
3. Why should cybersecurity experiments be conducted in isolated environments?
4. What could happen if penetration testing is performed on production systems?

21. Conclusion

In this lab, students successfully installed a virtualization platform and created the foundation of a cybersecurity laboratory environment. Kali Linux and Ubuntu Server virtual machines were deployed and configured to support future security experiments.

This virtual environment will be used in subsequent labs to deploy vulnerable web applications, perform vulnerability scanning, conduct penetration testing, and implement defensive security mechanisms.

The lab establishes the necessary infrastructure for hands-on cybersecurity learning and prepares students for advanced web application security exercises.

LAB 2

1. Lab Title: Virtual Network Configuration for Web Application Security Testing

2. Introduction

In cybersecurity laboratories, multiple systems must communicate with each other to simulate real-world network environments. For example, penetration testing typically involves an attacker machine interacting with a target server through a network.

Virtualization platforms such as VirtualBox allow users to create isolated virtual networks where virtual machines can communicate safely without affecting external networks. These virtual networks enable students to simulate real cyber attack scenarios in a controlled environment.

In this lab, students will configure a virtual network that connects two machines:

- Kali Linux: The attacker machine used for penetration testing and security analysis.
- Ubuntu Server: The target machine hosting vulnerable web applications.

Students will configure network adapters, assign static IP addresses, and verify network connectivity between the machines.

This virtual network will be used throughout the Web Application Security Lab Series for vulnerability scanning, web application attacks, and security defense experiments.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the concept of virtual networking in cybersecurity labs.
- Configure a Host-Only or Internal Network in VirtualBox.
- Connect multiple virtual machines to the same network.
- Assign static IP addresses to virtual machines.
- Verify network connectivity between attacker and target machines.

4. Significance of the Lab in Cybersecurity Training

Cybersecurity professionals must understand how systems communicate within networks. Most cyber-attacks occur across network connections where attackers send malicious traffic to target systems.

A properly configured lab network allows students to simulate realistic attack scenarios, such as:

- Web application attacks
- Vulnerability scanning
- Network reconnaissance

- Security monitoring

By learning how to configure virtual networks, students gain the ability to build secure testing environments that replicate real-world cybersecurity infrastructures.

5. Lab Scenario

A cybersecurity training environment requires two machines to communicate over a private network:

- An attacker machine (Kali Linux) used to perform security testing.
- A target machine (Ubuntu Server) hosting vulnerable applications.

Students must configure a virtual network in VirtualBox so these machines can communicate securely. The network will be isolated from the internet to prevent accidental attacks on external systems.

After configuring the network, students will assign IP addresses and verify connectivity using network diagnostic tools.

6. Lab Environment Architecture

The virtual lab network connects the attacker and target machines through an isolated internal network.

Network setup

Component	Role	IP Address	Description
VirtualBox Internal Network (LAN)	Network Environment	-	Provides isolated internal network
Kali Linux VM	Attacker	192.168.56.10	Security testing machine
Ubuntu Server VM	Target Web Server	192.168.56.20	Hosts web server for testing

Communication relationship

Source	Destination	Purpose
Kali Linux VM	Ubuntu Server VM	Security testing / attacks
Ubuntu Server VM	Kali Linux VM	Response to requests

7. Lab Requirements

Hardware Requirements

Minimum recommended system configuration:

Resource	Requirement
RAM	8 GB
Storage	40 GB free
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Attacker machine
Ubuntu Server	Target machine
Terminal tools	Network testing

8. Network Configuration Overview

The lab will use a Host-Only Network or Internal Network in VirtualBox.

Recommended network configuration:

Machine	IP Address
Kali Linux	192.168.56.10
Ubuntu Server	192.168.56.20

Subnet mask:

```
255.255.255.0
```

9. Step 1: Create Virtual Network in VirtualBox

Open VirtualBox Manager.

Navigate to:

```
File → Tools → Network Manager
```

Select:

```
Host-Only Networks
```

Click:

Create

VirtualBox will create a new network adapter.

Example:

vboxnet0

10. Step 2: Configure Network Adapter for Kali Linux

Select Kali Linux VM.

Open:

Settings → Network

Configure the adapter:

Parameter	Value
Adapter 1	Host-Only Adapter
Name	vboxnet0

Click OK.

11. Step 3: Configure Network Adapter for Ubuntu Server

Select Ubuntu Server VM.

Open:

Settings → Network

Configure the adapter:

Parameter	Value
Adapter 1	Host-Only Adapter
Name	vboxnet0

Click OK.

12. Step 4: Start Virtual Machines

Start both virtual machines:

1. Kali Linux
2. Ubuntu Server

Ensure both systems boot successfully.

13. Step 5: Configure Static IP Address on Ubuntu Server

Login to Ubuntu Server.

Assign an IP address:

192.168.56.20/24

```
ip a
```

14. Step 6: Configure IP Address on Kali Linux

Open a terminal in Kali Linux.

Assign an IP address:

192.168.56.10/24

Verify configuration:

```
ip a
```

15. Step 7: Test Network Connectivity

From Kali Linux, test communication with the Ubuntu server.

```
ping 192.168.56.20
```

Successful output example:

```
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=0.3 ms
```

Press:

```
CTRL + C
```

to stop the ping test.

16. Step 8: Verify Reverse Connectivity

From Ubuntu Server:

```
ping 192.168.56.10
```

Successful replies confirm two-way connectivity.

17. Expected Results

Students should observe:

Result	Description
Virtual network created	Host-only network operational
Kali Linux connected	Attacker machine network configured
Ubuntu Server connected	Target server network configured
Successful ping responses	Network connectivity confirmed

18. Deliverables

Students must submit screenshots showing:

1. VirtualBox network configuration
2. Kali Linux IP address configuration
3. Ubuntu Server IP configuration
4. Successful ping from Kali to Ubuntu
5. Successful ping from Ubuntu to Kali

19. Discussion Questions

1. What is the purpose of a virtual network in cybersecurity labs?
2. What is the difference between Host-Only Network and Internal Network?
3. Why should cybersecurity testing environments be isolated?
4. What problems could occur if IP addresses are misconfigured?

20. Conclusion

In this lab, students successfully configured a virtual network connecting two machines in an isolated cybersecurity testing environment. Static IP addresses were assigned, and network connectivity between Kali Linux and Ubuntu Server was verified.

This network infrastructure will be used in subsequent labs to deploy vulnerable web applications, perform penetration testing, conduct vulnerability scanning, and analyze security defenses.

The configured virtual network provides a safe and controlled environment for practicing real-world cybersecurity techniques.

LAB 3: Linux Server Administration Basics

1. Lab Title: Linux Server Administration for Cybersecurity Laboratories

2. Introduction

Linux operating systems play a critical role in cybersecurity environments. Most servers, cloud infrastructures, and security tools operate on Linux-based systems. Cybersecurity professionals must therefore possess strong Linux administration skills in order to deploy, configure, and secure systems.

In security testing environments, Linux servers are commonly used to host web applications, databases, and security monitoring tools. Administrators must be able to manage system packages, configure services, monitor system resources, and troubleshoot problems.

In this lab, students will practice essential Linux server administration tasks using the Ubuntu Server virtual machine created in the previous labs. Students will learn how to navigate the Linux file system, manage software packages, control system services, and monitor system status.

These fundamental skills are required for later labs where students will deploy web servers, install vulnerable web applications, and configure security tools.

3. Lab Objectives

By the end of this lab, students will be able to:

- Navigate the Linux file system using command-line tools.
- Manage software packages using the APT package manager.
- Install and update software packages.
- Manage system services using systemctl.
- Monitor system processes and system status.
- Create and manage user accounts.
- Prepare the Linux server for web application deployment.

4. Significance of the Lab in Cybersecurity Training

Linux systems are widely used in modern computing environments including web servers, cloud infrastructures, and enterprise networks. Security professionals must understand how Linux systems operate in order to secure them properly.

Many cybersecurity tools such as vulnerability scanners, intrusion detection systems, and forensic analysis tools run on Linux platforms. In addition, attackers frequently target Linux servers hosting web applications.

By mastering Linux server administration, students gain the foundational skills required to configure secure environments, analyze system activity, and respond to security incidents.

This lab provides the operational knowledge needed to deploy and manage the systems used in later cybersecurity exercises.

5. Lab Scenario

A university cybersecurity training lab uses an Ubuntu Server system to host web applications for security testing. Before deploying web servers and vulnerable applications, the system administrator must ensure that the server is properly configured and maintained.

Students will act as system administrators responsible for preparing the server environment. They will update the system, install software packages, monitor system processes, and manage system services.

These tasks are essential for maintaining secure and stable systems in real-world cybersecurity environments.

6. Lab Environment Architecture

The lab environment consists of two machines connected through the virtual network configured in Lab 2.

```
Cybersecurity Lab Network
       |
       |
+---------------------+        +----------------------+
| Kali Linux VM       |        | Ubuntu Server VM     |
| (Attacker Machine)  |        | (Target Server)      |
|                     |        |                      |
| Security Tools      |        | Linux Administration |
+---------------------+        +----------------------+
```

The focus of this lab is on the Ubuntu Server machine.

7. Lab Requirements

Hardware Requirements

Minimum recommended configuration:

Resource	Requirement
RAM	8 GB
Storage	40 GB
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Security testing system
Ubuntu Server	Linux server administration practice
Terminal	Command-line interface

8. Step 1: Login to Ubuntu Server

Start the Ubuntu Server virtual machine.

Login using the credentials created during installation.

Example:

```
Username: student
Password: ********
```

After login, the command prompt will appear.

Example prompt:

```
student@ubuntu-server:~$
```

9. Step 2: Explore the Linux File System

Display the current directory:

```
pwd
```

List files and directories:

```
ls
```

Display detailed information:

```
ls -l
```

Navigate between directories.

Example:

```
cd /
cd /home
cd /var
```

Return to the home directory:

```
cd ~
```

10. Step 3: View System Information

Check system information.

Display Linux distribution details:

```
lsb_release -a
```

Check system kernel version:

```
uname -r
```

Display system uptime:

```
uptime
```

Check memory usage:

```
free -h
```

11. Step 4: Update the System

Keeping systems updated is essential for security.

Update the package list:

```
sudo apt update
```

Upgrade installed packages:

```
sudo apt upgrade -y
```

This process downloads and installs the latest security updates.

12. Step 5: Install Software Packages

Install useful administrative tools.

Example installation:

```
sudo apt install net-tools -y
```

Verify installation:

```
ifconfig
```

Install another tool:

```
sudo apt install htop -y
```

Launch the system monitor:

```
htop
```

Press **Q** to exit.

13. Step 6: Manage System Services

Linux services are managed using the **systemctl** command.

Check system service status:

```
systemctl status ssh
```

Start a service:

```
sudo systemctl start ssh
```

Stop a service:

```
sudo systemctl stop ssh
```

Enable a service to start automatically:

```
sudo systemctl enable ssh
```

14. Step 7: Monitor Running Processes

View running processes:

```
ps aux
```

Display system processes interactively:

```
top
```

Press Q to exit.

These tools help administrators monitor system activity and detect suspicious processes.

15. Step 8: Manage Users

Create a new user account:

```
sudo adduser labuser
```

Follow the prompts to set a password.

Verify user creation:

```
cat /etc/passwd
```

Switch to the new user:

```
su - labuser
```

Return to the original user:

exit

16. Step 9: Check Network Configuration

Display network interfaces:

```
ip a
```

Check network connectivity:

```
ping 192.168.56.10
```

Press CTRL + C to stop the ping test.

17. Expected Results

Students should observe:

Result	Description
System information displayed	Linux server details verified
Software packages installed	Package manager working correctly
Services managed successfully	Services can be started and stopped
Processes monitored	System activity visible
Users created	User management functional
Network connectivity verified	Server connected to lab network

18. Deliverables

Students must submit screenshots showing:

1. Output of lsb_release -a
2. System update using sudo apt update
3. Installed package verification (htop)
4. Service status using systemctl
5. Created user account
6. Network connectivity test

19. Discussion Questions

1. Why is Linux widely used for web servers and cybersecurity tools?
2. What is the purpose of the APT package manager?
3. Why is it important to regularly update system packages?
4. What risks could occur if unauthorized users gain administrative access to a server?

20. Conclusion

In this lab, students practiced essential Linux server administration skills required for managing cybersecurity environments. Students explored the Linux file system, updated system packages, installed software tools, managed system services, monitored system processes, and created user accounts.

These fundamental administration skills prepare students for the next stage of the Web Application Security Lab Series, where they will deploy a web server environment and host vulnerable web applications for security testing.

Mastery of Linux administration enables cybersecurity professionals to configure secure systems, maintain operational environments, and analyze potential security incidents.

LAB 4: Deploying the Apache Web Server Environment (LAMP Stack)

1. Lab Title: Deploying a LAMP Stack (Linux, Apache, MySQL, PHP) for Web Application Hosting

2. Introduction

Modern web applications rely on server environments that support web services, database management, and dynamic content processing. One of the most widely used web server environments is the LAMP stack, which consists of:

- Linux: the operating system that hosts the server.
- Apache: the web server that handles HTTP requests.
- MySQL/MariaDB: the database management system.
- PHP: the server-side scripting language used to generate dynamic web pages.

The LAMP stack is widely used to host websites, enterprise applications, and web services. Many popular platforms such as WordPress, Drupal, and Moodle run on this architecture.

In cybersecurity training environments, LAMP servers are often used to host intentionally vulnerable applications that allow students to study web security vulnerabilities.

In this lab, students will install and configure the LAMP stack on the Ubuntu Server machine. Once installed, students will verify the web server functionality and prepare the system for hosting a vulnerable web application in the next lab.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the architecture of a LAMP stack.
- Install and configure the Apache web server.
- Install and configure the MySQL database server.
- Install PHP and required extensions.
- Verify the functionality of the web server.
- Prepare the environment for web application deployment.

4. Significance of the Lab in Cybersecurity Training

Many cyber-attacks target web servers and web applications hosted on internet-facing infrastructure. Understanding how web servers are deployed and configured is essential for cybersecurity professionals.

Security analysts, penetration testers, and system administrators must understand how web services operate in order to identify vulnerabilities and implement security controls.

By deploying a LAMP stack, students gain practical knowledge of how web applications interact with web servers and databases. This understanding is critical for performing vulnerability scanning, penetration testing, and implementing defensive security measures.

This lab provides the necessary infrastructure for future cybersecurity labs involving vulnerable web applications and web application security testing.

5. Lab Scenario

A university cybersecurity lab requires a web server to host applications used for security training. The system administrator must deploy a web server environment capable of running dynamic web applications.

Students will configure the Ubuntu Server system as a web server by installing the LAMP stack. The server will host web applications that students will later analyze for vulnerabilities and security weaknesses.

Once the LAMP stack is installed and configured, the server will be ready for deployment of the DVWA vulnerable web application used in subsequent security labs.

6. Lab Environment Architecture

The lab environment consists of two machines connected through the virtual network configured in previous labs.

```
Cybersecurity Lab Network
        |
        |
+---------------------+       +---------------------------+
| Kali Linux VM       |       | Ubuntu Server VM          |
| (Attacker Machine)  |       | (Web Server Environment)  |
|                     |       |                           |
| Security Tools      | ----> | Apache Web Server         |
| Browser Access      |       | PHP Engine                |
|                     |       | MySQL Database            |
+---------------------+       +---------------------------+
```

The Ubuntu server hosts the LAMP stack, while Kali Linux accesses the web server using a browser.

7. Lab Requirements

Hardware Requirements

Minimum recommended system configuration:

Resource	Requirement
RAM	8 GB
Storage	40 GB
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Web client for testing
Ubuntu Server	Web server environment
Apache	Web server software
MySQL	Database server
PHP	Server-side scripting language

8. Step 1: Update the System

Before installing new software, update the system packages.

On the Ubuntu server, run:

```
sudo apt update
sudo apt upgrade -y
```

Updating ensures the latest security patches and package versions are installed.

9. Step 2: Install Apache Web Server

Install the Apache web server using the package manager.

```
sudo apt install apache2 -y
```

Start the Apache service:

```
sudo systemctl start apache2
```

Enable Apache to start automatically at system boot:

```
sudo systemctl enable apache2
```

Verify Apache service status:

```
sudo systemctl status apache2
```

10. Step 3: Test Apache Web Server

Open a web browser on the Kali Linux machine.

Navigate to the Ubuntu server IP address:

```
http://192.168.56.20
```

If the installation was successful, the Apache default web page should appear.

This confirms that the web server is running and accessible over the network.

11. Step 4: Install MySQL Database Server

Install the MySQL database server.

```
sudo apt install mysql-server -y
```

Verify MySQL service status:

```
sudo systemctl status mysql
```

Run the security configuration script:

```
sudo mysql_secure_installation
```

During configuration:

- Set root password
- Remove anonymous users
- Disable remote root login
- Remove test database

These steps improve database security.

12. Step 5: Install PHP and Required Extensions

Install PHP and required extensions for web applications.

```
sudo apt install php libapache2-mod-php php-mysql php-cli php-curl php-gd php-xml php-mbstring -y
```

Restart Apache to apply PHP support.

```
sudo systemctl restart apache2
```

Verify PHP installation:

```
php -v
```

13. Step 6: Create a PHP Test Page

Navigate to the web server root directory.

```
cd /var/www/html
```

Create a test PHP file.

```
sudo nano info.php
```

Add the following code:

```
<?php
phpinfo();
?>
```

Save and exit.

14. Step 7: Verify PHP Processing

Open the browser in Kali Linux and visit:

```
http://192.168.56.20/info.php
```

The PHP information page should appear, showing the installed PHP version and modules.

This confirms that Apache is correctly processing PHP scripts.

15. Step 8: Verify Database Access

Login to the MySQL database server.

```
sudo mysql
```

Display database information.

```
SHOW DATABASES;
```

Exit MySQL:

EXIT;

16. Expected Results

Students should observe:

Result	Description
Apache installed	Web server operational
Web page accessible	Server reachable from Kali Linux
PHP functioning	Dynamic web pages processed
MySQL running	Database service operational
LAMP stack operational	Environment ready for web applications

17. Deliverables

Students must submit the following screenshots:

1. Apache service status
2. Apache default web page in browser
3. PHP info page output
4. MySQL login session
5. Installed PHP version

18. Discussion Questions

1. What components make up the LAMP stack?
2. What is the role of Apache in a web server environment?
3. Why do web applications require a database server?
4. What risks could occur if web servers are misconfigured?

19. Conclusion

In this lab, students successfully deployed a LAMP stack on the Ubuntu Server system. Apache, MySQL, and PHP were installed and configured to support dynamic web applications.

The web server was tested from the Kali Linux machine to confirm network connectivity and application functionality. A PHP test page verified that the server can process dynamic scripts.

This environment provides the foundation for the next lab, where students will deploy a vulnerable web application (DVWA) and begin performing web application security testing.

LAB 5

1. Lab Title: Deploying the Damn Vulnerable Web Application (DVWA) for Web Application Security Testing

2. Introduction

Web applications are frequently targeted by cyber attackers because they often process user input, interact with databases, and are accessible through the internet. Security professionals must therefore understand how web applications function and how vulnerabilities can be exploited.

To safely study web application security, cybersecurity training environments often use intentionally vulnerable applications. One of the most widely used training platforms is the Damn Vulnerable Web Application (DVWA).

DVWA is a PHP/MySQL web application designed to demonstrate common web vulnerabilities such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- File Inclusion
- Brute Force attacks

These vulnerabilities allow students to practice penetration testing techniques in a controlled environment.

In this lab, students will deploy DVWA on the Ubuntu server configured in the previous lab. Once deployed, students will verify that the application is operational and configure the security level to allow vulnerability testing.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the purpose of vulnerable web applications in cybersecurity training.
- Download and deploy DVWA on a Linux web server.
- Configure the database required for DVWA.
- Configure file permissions and application settings.
- Verify successful deployment of the vulnerable application.
- Prepare the system for web application security testing.

4. Significance of the Lab in Cybersecurity Training

Learning about web vulnerabilities through theory alone is not sufficient. Security professionals must practice identifying and exploiting vulnerabilities in controlled environments.

DVWA allows students to interact with intentionally vulnerable code and understand how attacks work in real-world scenarios. Through hands-on exercises, students gain practical experience in vulnerability scanning, exploitation techniques, and security defense mechanisms.

This lab provides the platform used in later labs involving tools such as OWASP ZAP, Burp Suite, and Web Application Firewalls.

5. Lab Scenario

A cybersecurity training environment requires a vulnerable web application that students can analyze for security weaknesses. The system administrator has prepared a web server using the LAMP stack.

Students must deploy the Damn Vulnerable Web Application (DVWA) on the Ubuntu server. Once installed, the application will be accessed from the Kali Linux machine to verify that it is working correctly.

This vulnerable application will serve as the target system for future security testing labs.

6. Lab Environment Architecture

The lab environment consists of two machines connected through the virtual network.

```
Cybersecurity Lab Network
       |
       |
+---------------------+       +---------------------------+
| Kali Linux VM       |       | Ubuntu Server VM          |
| (Attacker Machine)  |       | (Web Application Server)  |
|                     |       |                           |
| Web Browser         | ----> | Apache Web Server         |
| Security Tools      |       | DVWA Application          |
|                     |       | MySQL Database            |
+---------------------+       +---------------------------+
```

Kali Linux will access DVWA using a web browser.

7. Lab Requirements

Hardware Requirements

Resource	Requirement
RAM	8 GB
Storage	40 GB
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Attacker machine
Ubuntu Server	Web server
Apache	Web server software
MySQL	Database server
PHP	Server-side scripting
Git	Download DVWA source code

8. Step 1: Verify Web Server Environment

Ensure that the LAMP stack installed in the previous lab is functioning.

Check Apache status:

```
sudo systemctl status apache2
```

Check MySQL status:

```
sudo systemctl status mysql
```

If services are not running, start them:

```
sudo systemctl start apache2
sudo systemctl start mysql
```

9. Step 2: Install Git

Git is required to download the DVWA source code.

Install Git:

```
sudo apt install git -y
```

Verify installation:

```
git --version
```

10. Step 3: Download DVWA Source Code

Navigate to the Apache web directory:

```
cd /var/www/html
```

Clone the DVWA repository:

```
sudo git clone https://github.com/digininja/DVWA.git
```

Verify that the DVWA directory has been created:

```
ls
```

You should see the DVWA folder in the directory listing.

11. Step 4: Configure File Permissions

Set proper ownership for the web server.

```
sudo chown -R www-data:www-data /var/www/html/DVWA
```

Set directory permissions:

```
sudo chmod -R 755 /var/www/html/DVWA
```

These permissions allow the web server to access the application files.

12. Step 5: Create DVWA Database

Login to MySQL:

```
sudo mysql
```

Create the DVWA database:

```
CREATE DATABASE dvwa;
```

Create a database user:

```
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'password';
```

Grant privileges:

```
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
```

Apply changes:

```
FLUSH PRIVILEGES;
```

Exit MySQL:

```
EXIT;
```

13. Step 6: Configure DVWA Database Settings

Navigate to the DVWA configuration directory.

```
cd /var/www/html/DVWA/config
```

Copy the configuration template:

```
sudo cp config.inc.php.dist config.inc.php
```

Edit the configuration file:

```
sudo nano config.inc.php
```

Update the database settings:

```
$_DVWA['db_user'] = 'dvwa';
$_DVWA['db_password'] = 'password';
$_DVWA['db_database'] = 'dvwa';
```

Save and exit.

14. Step 7: Restart Apache Web Server

Restart the Apache service to apply configuration changes.

```
sudo systemctl restart apache2
```

15. Step 8: Access DVWA from Kali Linux

Open a web browser on the Kali Linux machine.

Navigate to the DVWA application:

```
http://192.168.56.20/DVWA
```

You should see the DVWA Setup Page.

Click:

```
Create / Reset Database
```

16. Step 9: Login to DVWA

Use the default credentials:

Field	Value
Username	admin
Password	password

After logging in, the DVWA dashboard will appear.

17. Step 10: Configure Security Level

Navigate to the DVWA Security tab.

Set the security level to:

```
Low
```

This allows vulnerabilities to be easily demonstrated during testing.

18. Expected Results

Students should observe:

Result	Description
DVWA webpage loads	Application deployed successfully
Database connection established	Application communicates with MySQL
Login successful	Default credentials functional
Vulnerability modules available	Environment ready for testing

19. Deliverables

Students must submit screenshots showing:

1. DVWA setup page
2. Database creation page
3. DVWA login page
4. DVWA dashboard
5. DVWA security level configuration

20. Discussion Questions

1. Why are vulnerable applications used in cybersecurity training?
2. What risks would occur if DVWA were exposed to the internet?
3. What components are required to run a dynamic web application?
4. Why should penetration testing be conducted only in controlled environments?

21. Conclusion

In this lab, students successfully deployed the Damn Vulnerable Web Application (DVWA) on the Ubuntu server using the LAMP stack environment. The application was configured to communicate with the MySQL database and was accessed successfully from the Kali Linux machine.

This vulnerable web application provides a safe platform for practicing web application security testing techniques. In subsequent labs, students will use security tools to identify and exploit vulnerabilities present in DVWA.

The environment created in this lab forms the foundation for advanced web security experiments involving vulnerability scanning, penetration testing, and defensive security mechanisms.

LAB 6: HTTP Traffic Analysis Using Browser Developer Tools and Burp Suite

1. Analyzing HTTP Requests and Responses Using Browser Developer Tools and Burp Suite

2. Introduction

Web applications communicate with clients through the **Hypertext Transfer Protocol (HTTP)**. Each time a user interacts with a website such as logging in, submitting a form, or clicking a link the browser sends HTTP requests to the web server, which responds with HTTP responses.

Understanding HTTP communication is essential for cybersecurity professionals because many web application attacks exploit weaknesses in how servers process these requests.

Security testing tools such as **Burp Suite** allow analysts to intercept, inspect, and modify HTTP traffic between a client and a server. By analyzing HTTP requests and responses, security professionals can identify vulnerabilities such as SQL injection, cross-site scripting, and insecure authentication mechanisms.

In this lab, students will learn how HTTP communication works by analyzing web traffic generated when interacting with the DVWA application. Students will first inspect HTTP requests using **browser developer tools**, then intercept and analyze the same traffic using **Burp Suite**.

This knowledge will prepare students for advanced web application security testing labs.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the structure of HTTP requests and responses.
- Analyze web traffic using browser developer tools.
- Identify HTTP methods such as GET and POST.
- Intercept HTTP traffic using Burp Suite.
- Examine HTTP headers, cookies, and parameters.
- Understand how user input is transmitted to web servers.

4. Significance of the Lab in Cybersecurity Training

Many web attacks exploit weaknesses in how web applications process user input. By understanding how browsers communicate with servers, cybersecurity professionals can identify vulnerabilities in authentication mechanisms, form submissions, and session management.

HTTP traffic analysis is an essential skill for penetration testers and security analysts. Tools such as Burp Suite allow professionals to inspect application behavior and identify security flaws.

This lab provides the foundational knowledge required to perform vulnerability scanning and manual exploitation in later labs.

5. Lab Scenario

A cybersecurity analyst is investigating a web application to understand how it processes user requests. The analyst must examine the communication between the browser and the server to identify potential vulnerabilities.

Students will analyze HTTP traffic generated when interacting with the **DVWA application** hosted on the Ubuntu server. Using browser developer tools and Burp Suite, students will observe how login requests and other interactions are transmitted to the server.

Understanding this communication process allows analysts to identify weaknesses that attackers could exploit.

6. Lab Environment Architecture

The environment consists of a web application server and a client machine intercepting HTTP traffic.

```
Cybersecurity Lab Network

+---------------------+          +---------------------------+
| Kali Linux VM       |          | Ubuntu Server VM          |
|                     |          |                           |
| Web Browser         | -------> | Apache Web Server         |
| Burp Suite Proxy    |          | DVWA Web Application      |
|                     |          | MySQL Database            |
+---------------------+          +---------------------------+
```

Burp Suite acts as a proxy between the browser and the server.

7. Lab Requirements

Hardware Requirements

Resource	Requirement
RAM	8 GB
Storage	40 GB
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Security testing machine
Ubuntu Server	DVWA web server
Web Browser	Web client
Burp Suite	HTTP interception tool
DVWA	Vulnerable web application

8. Step 1: Access DVWA Application

Open a browser on the Kali Linux machine.

Navigate to the DVWA application:

```
http://192.168.56.20/DVWA
```

Login using the default credentials:

Field	Value
Username	admin
Password	password

Once logged in, the DVWA dashboard will appear.

9. Step 2: Open Browser Developer Tools

Browser developer tools allow users to inspect network traffic.

In Firefox or Chromium:

Press:

```
F12
```

Open the Network tab.

```
Reload the DVWA page.
```

The network panel will display all HTTP requests sent by the browser.

10. Step 3: Analyze HTTP Requests

Click one of the HTTP requests shown in the network panel.

Observe the request details.

Example request:

```
GET /DVWA/index.php HTTP/1.1
Host: 192.168.56.20
User-Agent: Mozilla/5.0
```

Important HTTP request components include:

Component	Description
Method	Action performed (GET, POST)
URL	Resource requested
Headers	Additional request information
Cookies	Session data

11. Step 4: Analyze Login Request

Logout of DVWA and login again while monitoring the network panel.

Identify the login request.

Example request:

```
POST /DVWA/login.php HTTP/1.1
Host: 192.168.56.20
username=admin&password=password
```

Observe how the username and password are transmitted to the server.

12. Step 5: Launch Burp Suite

Open Burp Suite in Kali Linux.

Run:

```
burpsuite
```

When Burp Suite starts:

1. Select Temporary Project
2. Click Next

3. Choose Use Burp Defaults
4. Click Start Burp

13. Step 6: Configure Burp Proxy

Burp Suite runs a proxy server at:

```
127.0.0.1:8080
```

Enable request interception.

Navigate to:

```
Proxy → Intercept
```

Click:

```
Intercept ON
```

14. Step 7: Configure Browser Proxy

Open browser settings.

Navigate to:

```
Settings → Network Settings
```

Configure proxy:

Parameter	Value
HTTP Proxy	127.0.0.1
Port	8080

Enable:

```
Use this proxy for all protocols
```

Save settings.

15. Step 8: Intercept HTTP Requests

Return to the DVWA application.

Refresh the page.

Burp Suite will intercept the HTTP request.

Example intercepted request:

```
GET /DVWA/index.php HTTP/1.1
Host: 192.168.56.20
User-Agent: Mozilla/5.0
```

Click:

Forward

to send the request to the server.

16. Step 9: Inspect Request Parameters

Navigate to the DVWA SQL Injection module.

Enter:

1

Burp Suite intercepts the request.

Example request:

GET /DVWA/vulnerabilities/sqli/?id=1&Submit=Submit

Observe the parameter:

id=1

This parameter will later be used in attack labs.

17. Step 10: Disable Proxy After Testing

Once analysis is complete:

Disable interception in Burp Suite.

Navigate to:

Proxy → Intercept OFF

Reset browser proxy settings to No Proxy.

18. Expected Results

Students should observe:

Result	Description
HTTP requests captured	Browser developer tools functioning
Login request analyzed	Form data visible in HTTP request
Burp proxy intercepts traffic	HTTP requests intercepted successfully
Request parameters identified	Inputs visible for testing

19. Deliverables

Students must submit screenshots showing:

1. Browser developer tools network panel
2. Example HTTP request captured
3. DVWA login request details
4. Burp Suite intercepting HTTP request
5. HTTP request showing parameters

20. Discussion Questions

1. What is the difference between HTTP GET and POST requests?
2. Why is it important to understand HTTP request structure during security testing?
3. What role does Burp Suite play in web application penetration testing?
4. What security risks exist if sensitive data is transmitted without encryption?

21. Conclusion

In this lab, students analyzed HTTP communication between a web browser and the DVWA web server. Using browser developer tools and Burp Suite, students inspected HTTP requests, examined parameters, and observed how user input is transmitted to web applications.

Understanding HTTP traffic is a fundamental skill for cybersecurity professionals, as many web vulnerabilities arise from improper handling of user input within HTTP requests.

The knowledge gained in this lab prepares students for the next stage of the Web Application Security Lab Series, where they will use automated tools and manual techniques to identify and exploit vulnerabilities in the DVWA application.

LAB 7: Web Application Vulnerability Scanning Using OWASP ZAP

1. Scanning Web Applications for Vulnerabilities Using OWASP ZAP

2. Introduction

Web applications often contain security vulnerabilities that attackers can exploit to gain unauthorized access, manipulate data, or compromise systems. Identifying these vulnerabilities before attackers do is a critical responsibility of cybersecurity professionals.

Vulnerability scanning tools are commonly used to detect potential weaknesses in web applications automatically. One of the most widely used open-source tools for web application security testing is OWASP ZAP (Zed Attack Proxy).

OWASP ZAP is designed to identify common web application vulnerabilities such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- Insecure cookies
- Information disclosure

The tool works by intercepting and analyzing HTTP traffic between a client and a web application. It can perform passive scanning, which observes normal traffic, and active scanning, which attempts to exploit vulnerabilities.

In this lab, students will use OWASP ZAP to scan the Damn Vulnerable Web Application (DVWA) deployed in the previous lab. Students will discover vulnerabilities, analyze scan results, and generate vulnerability reports.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the role of vulnerability scanners in web application security.
- Launch and configure OWASP ZAP.
- Configure a browser proxy for ZAP.
- Perform passive vulnerability scanning.
- Perform active vulnerability scanning.

- Identify common web application vulnerabilities.
- Generate vulnerability scan reports.

4. Significance of the Lab in Cybersecurity Training

Automated vulnerability scanning is an important step in the cybersecurity assessment process. Security teams often use scanners to identify potential weaknesses in applications before conducting deeper manual penetration testing.

Tools such as OWASP ZAP allow security analysts to detect common vulnerabilities quickly and efficiently. However, automated scanners are not perfect and must be used alongside manual testing techniques.

This lab introduces students to the automated vulnerability discovery process and prepares them for the next stage of security testing, where vulnerabilities will be manually exploited using tools such as Burp Suite.

5. Lab Scenario

A cybersecurity analyst is responsible for assessing the security of a web application hosted on an internal server. Before performing manual penetration testing, the analyst uses a vulnerability scanner to identify potential weaknesses.

Students will use OWASP ZAP to scan the DVWA application hosted on the Ubuntu server. The tool will automatically analyze the application, discover pages, and test for vulnerabilities.

The results of the scan will help identify security weaknesses that attackers might exploit.

6. Lab Environment Architecture

The vulnerability scanning environment consists of a scanning machine and a target web server.

```
Cybersecurity Lab Network

+---------------------+          +---------------------------+
| Kali Linux VM       |          | Ubuntu Server VM          |
|                     |          |                           |
| OWASP ZAP Scanner   | -------> | Apache Web Server         |
| Web Browser         |          | DVWA Web Application      |
|                     |          | MySQL Database            |
+---------------------+          +---------------------------+
```

OWASP ZAP intercepts and analyzes HTTP traffic between the browser and the DVWA web server.

7. Lab Requirements

Hardware Requirements

Resource	Requirement
RAM	8 GB
Storage	40 GB
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Security testing machine
Ubuntu Server	Web application server
DVWA	Vulnerable web application
OWASP ZAP	Vulnerability scanning tool
Web Browser	Access web application

8. Step 1: Verify DVWA Application

Open a browser in **Kali Linux**.

Navigate to the DVWA application:

```
http://192.168.56.20/DVWA
```

Login with the default credentials:

Field	Value
Username	admin
Password	password

Verify that the DVWA dashboard loads successfully.

9. Step 2: Launch OWASP ZAP

Open a terminal in Kali Linux and run:

```
zaproxy
```

When ZAP starts:

1. Select No, I do not want to persist this session
2. Click Start

The OWASP ZAP dashboard will appear.

10. Step 3: Configure ZAP Proxy

OWASP ZAP runs a local proxy service.

Default proxy address:

```
127.0.0.1
```

Default port:

```
8080
```

This proxy will intercept HTTP traffic between the browser and the web server.

11. Step 4: Configure Browser Proxy

Open the browser network settings.

Navigate to:

```
Settings → Network Settings
```

Configure the proxy settings.

Parameter	Value
HTTP Proxy	127.0.0.1
Port	8080

Enable:

```
Use this proxy for all protocols
```

Save the configuration.

12. Step 5: Access DVWA Through ZAP

Return to the browser and open the DVWA application again:

```
http://192.168.56.20/DVWA
```

Login to the application.

OWASP ZAP will automatically begin passive scanning by analyzing the HTTP traffic.

13. Step 6: Explore the Discovered Application Structure

In the ZAP interface, open the Sites panel.

Expand the discovered site:

```
http://192.168.56.20
```

You should see multiple application components, such as:

- login.php
- index.php
- vulnerabilities directory
- security settings page

This represents the structure of the DVWA web application.

14. Step 7: Perform Spider Scan

Spider scanning discovers all accessible pages in the web application.

Right-click the target site in the Sites panel.

Select:

```
Attack → Spider
```

Click:

```
Start Scan
```

The spider will crawl the application and identify additional pages and parameters.

15. Step 8: Perform Active Vulnerability Scan

Active scanning attempts to identify vulnerabilities by sending attack payloads.

Right-click the target site again.

Select:

Attack → Active Scan

Click:

Start Scan

ZAP will test the application for vulnerabilities such as:

- SQL Injection
- Cross-Site Scripting
- Command Injection
- Directory traversal

16. Step 9: Monitor Scan Progress

Open the Active Scan tab.

Observe the following information:

- Requests sent
- Payloads used
- Scan progress percentage
- Detected issues

Scanning may take several minutes depending on the application size.

17. Step 10: View Detected Vulnerabilities

Detected vulnerabilities appear in the Alerts panel.

Example vulnerabilities:

Vulnerability	Risk Level
Cross-Site Scripting	High
SQL Injection	High
Cookie without Secure Flag	Medium
Information Disclosure	Low

Click any alert to view detailed information including:

- Vulnerability description
- Evidence
- Risk level
- Recommended mitigation

18. Step 11: Generate Vulnerability Scan Report

Generate a report of the scan results.

Navigate to:

```
Report → Generate Report
```

Choose the report format:

- HTML
- PDF
- XML

Save the report.

Example filename:

```
dvwa_scan_report.html
```

19. Expected Results

Students should observe:

Result	Description
OWASP ZAP intercepts traffic	Proxy configuration successful
Spider scan discovers pages	Application structure identified
Active scan launches attacks	Vulnerability testing performed
Alerts generated	Vulnerabilities detected

20. Deliverables

Students must submit screenshots showing:

1. OWASP ZAP dashboard
2. Spider scan results
3. Active scan progress
4. Vulnerability alerts panel
5. Generated vulnerability scan report

21. Discussion Questions

1. What is the difference between passive scanning and active scanning?
2. Why should vulnerability scanners be used carefully on production systems?
3. What limitations exist with automated vulnerability scanning tools?
4. Why is manual penetration testing still required after automated scanning?

22. Conclusion

In this lab, students used OWASP ZAP to automatically scan the DVWA web application for security vulnerabilities. The tool discovered application pages, analyzed HTTP traffic, and attempted to identify common web application weaknesses.

Automated vulnerability scanning is an important step in the security assessment process. However, it should always be combined with manual testing to identify more complex vulnerabilities.

The results of this scan provide a starting point for deeper security analysis. In the next lab, students will manually exploit vulnerabilities using **Burp Suite** to better understand how attackers manipulate HTTP requests to compromise web applications.

LAB 8: Performing Web Application Attacks Using Burp Suite (SQL Injection and XSS)

1. Exploiting Web Application Vulnerabilities Using Burp Suite

2. Introduction

After identifying potential vulnerabilities using automated scanning tools, cybersecurity professionals often perform manual penetration testing to confirm and exploit those weaknesses. Manual testing allows security analysts to interact directly with web application traffic and manipulate requests sent to the server.

One of the most widely used tools for manual web application penetration testing is Burp Suite. Burp Suite functions as an intercepting proxy, allowing analysts to capture, inspect, and modify HTTP requests between a browser and a web server.

Attackers commonly exploit vulnerabilities by manipulating parameters within HTTP requests. Two of the most common web application attacks include:

- SQL Injection: manipulating database queries to retrieve unauthorized data.
- Cross-Site Scripting (XSS): injecting malicious scripts that execute in the user's browser.

In this lab, students will use Burp Suite to intercept and modify HTTP requests sent to the Damn Vulnerable Web Application (DVWA). By altering request parameters, students will demonstrate how SQL injection and XSS attacks can compromise vulnerable web applications.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the role of Burp Suite in web application penetration testing.
- Configure a browser proxy for Burp Suite.
- Intercept HTTP requests between the client and server.
- Modify HTTP request parameters.
- Perform SQL Injection attacks.
- Perform Cross-Site Scripting (XSS) attacks.
- Analyze server responses to determine vulnerability.

4. Significance of the Lab in Cybersecurity Training

Manual penetration testing provides deeper insight into how web applications process user input and how attackers manipulate HTTP requests to exploit vulnerabilities.

While automated tools such as OWASP ZAP can identify many vulnerabilities, manual testing allows security professionals to verify vulnerabilities, craft custom payloads, and analyze application behavior in detail.

Burp Suite is widely used by penetration testers and security researchers for web application security testing. Learning how to use this tool helps students develop practical skills required in real-world cybersecurity roles.

This lab demonstrates how attackers exploit common web vulnerabilities and prepares students to understand defensive measures implemented in later labs.

5. Lab Scenario

A cybersecurity analyst has identified potential vulnerabilities in a web application using an automated vulnerability scanner. To verify these findings, the analyst performs manual penetration testing using Burp Suite.

Students will intercept HTTP requests sent to the DVWA web application and modify request parameters to simulate attacks. By manipulating input fields, students will observe how vulnerable applications process malicious inputs.

The results of these attacks will demonstrate the importance of secure coding practices and proper input validation.

6. Lab Environment Architecture

The lab environment consists of a client system intercepting and modifying HTTP requests sent to a vulnerable web application.

```
Cybersecurity Lab Network

+---------------------+          +---------------------------+
| Kali Linux VM       |          | Ubuntu Server VM          |
|                     |          |                           |
| Web Browser         | -------> | Apache Web Server         |
| Burp Suite Proxy    |          | DVWA Web Application      |
|                     |          | MySQL Database            |
+---------------------+          +---------------------------+
```

Burp Suite intercepts HTTP traffic between the browser and the DVWA web server.

7. Lab Requirements

Hardware Requirements

Resource	Requirement
RAM	8 GB
Storage	40 GB
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Security testing machine
Ubuntu Server	Web application server
DVWA	Vulnerable web application
Burp Suite	Web application testing tool
Web Browser	Client for interacting with the application

8. Step 1: Launch Burp Suite

Open a terminal in Kali Linux and run:

```
burpsuite
```

When Burp Suite launches:

1. Select Temporary Project
2. Click Next
3. Select Use Burp Defaults
4. Click Start Burp

The Burp Suite dashboard will appear.

9. Step 2: Configure Burp Proxy

Burp Suite uses a local proxy server to intercept HTTP traffic.

Default proxy configuration:

```
IP Address: 127.0.0.1
Port: 8080
```

Navigate to:

```
Proxy → Intercept
```

Enable interception by clicking:

```
Intercept ON
```

10. Step 3: Configure Browser Proxy

Open the browser network settings.

Navigate to:

```
Settings → Network Settings
```

Configure the proxy settings:

Parameter	Value
HTTP Proxy	127.0.0.1
Port	8080

Enable:

```
Use this proxy for all protocols
```

Save the configuration.

11. Step 4: Intercept Web Traffic

Open the DVWA application in the browser:

```
http://192.168.56.20/DVWA
```

Login with the credentials:

Field	Value
Username	admin
Password	password

Burp Suite will intercept the login request.

Example intercepted request:

```
POST /DVWA/login.php HTTP/1.1
Host: 192.168.56.20
username=admin&password=password
```

Click Forward to allow the request to proceed.

12. Step 5: Perform SQL Injection Attack

Navigate to the DVWA module:

```
DVWA → SQL Injection
```

Enter the value:

```
1
```

Submit the request.

Burp Suite will intercept the request.

Example request:

```
GET /DVWA/vulnerabilities/sqli/?id=1&Submit=Submit
```

Right-click the request and select:

```
Send to Repeater
```

13. Step 6: Modify SQL Injection Payload

Modify the parameter:

```
id=1
```

Replace with:

```
id=1 OR 1=1
```

Send the modified request.

If the application is vulnerable, the database query will return multiple records, indicating a successful SQL injection attack.

14. Step 7: Perform Cross-Site Scripting Attack

Navigate to the DVWA module:

```
DVWA → XSS (Reflected)
```

Enter the following input:

```
test
```

Intercept the request using Burp Suite.

Send the request to Repeater.

Replace the parameter with the payload:

```
<script>alert('XSS')</script>
```

Send the request.

If successful, the browser will display a popup alert indicating that the injected script has executed.

15. Step 8: Use Burp Intruder for Automated Testing

Right-click the intercepted request and select:

Send to Intruder

Configure the payload position:

```
id=§1§
```

Add a list of SQL injection payloads.

Start the attack.

Burp Intruder will automatically test multiple payloads to identify vulnerabilities.

16. Step 9: Analyze Server Responses

Observe the server responses to determine whether the application is vulnerable.

Indicators of vulnerability include:

Indicator	Description
Database errors	SQL injection vulnerability
Unexpected data returned	Query manipulation
Script execution	XSS vulnerability

17. Step 10: Reset Browser Proxy Settings

After completing the test:

Disable Burp interception:

```
Proxy → Intercept OFF
```

Reset browser proxy settings to No Proxy.

18. Expected Results

Students should observe:

Result	Description
HTTP requests intercepted	Burp proxy functioning
Request parameters modified	Manual manipulation possible
SQL injection payload executed	Database query manipulated
XSS payload executed	JavaScript executed in browser

19. Deliverables

Students must submit screenshots showing:

1. Burp Suite intercepting an HTTP request
2. SQL injection payload in Burp Repeater
3. Successful SQL injection result
4. XSS alert popup
5. Burp Intruder attack results

20. Discussion Questions

1. How does SQL injection allow attackers to access database information?
2. What security practices can prevent SQL injection attacks?
3. How does Cross-Site Scripting affect web application users?
4. Why is manual penetration testing necessary even after automated scanning?

21. Conclusion

In this lab, students performed manual web application penetration testing using Burp Suite. By intercepting and modifying HTTP requests, students demonstrated how SQL injection and Cross-Site Scripting attacks exploit vulnerabilities in web applications.

These exercises highlight the importance of secure coding practices, input validation, and proper application security testing.

Understanding how attackers manipulate HTTP requests allows cybersecurity professionals to design more secure web applications and implement effective defensive controls.

LAB 9: Implementing a Web Application Firewall (WAF) Using ModSecurity

1. Implementing a Web Application Firewall (WAF) Using ModSecurity to Protect Web Applications

2. Introduction

Web applications are frequently targeted by cyber attackers because they interact with users, process input data, and connect to backend databases. Vulnerabilities in web applications can allow attackers to perform malicious actions such as stealing data, modifying application behavior, or gaining unauthorized access.

To protect web applications from these threats, organizations often deploy a Web Application Firewall (WAF). A WAF monitors and filters HTTP traffic between users and web servers, detecting malicious requests and blocking attacks before they reach the application.

One widely used open-source WAF is ModSecurity, which integrates with web servers such as Apache and Nginx. ModSecurity uses a rule-based system to detect malicious patterns in HTTP requests and responses.

To enhance its detection capabilities, ModSecurity can use the OWASP Core Rule Set (CRS). The CRS provides a comprehensive set of security rules designed to detect common web application attacks including SQL injection, cross-site scripting, command injection, and file inclusion attacks.

In this lab, students will install and configure ModSecurity on the Apache web server hosting DVWA. Students will then perform web attacks and observe how the WAF detects and blocks malicious requests.

3. Lab Objectives

By the end of this lab, students will be able to:

- Understand the purpose and function of a Web Application Firewall.
- Install and configure ModSecurity on an Apache web server.
- Install and configure the OWASP Core Rule Set.
- Enable the ModSecurity rule engine.
- Perform SQL injection and XSS attacks against the web application.
- Analyze WAF logs to observe attack detection.

4. Significance of the Lab in Cybersecurity Training

While previous labs focused on identifying and exploiting vulnerabilities, cybersecurity professionals must also understand how to defend systems against these attacks.

Web Application Firewalls are widely used in enterprise environments to protect web servers from malicious traffic. By deploying and configuring a WAF, security administrators can prevent common web attacks even when applications contain vulnerabilities.

This lab introduces students to defensive security technologies and demonstrates how security tools detect and block attacks in real time. Understanding WAF operation is essential for professionals involved in web application security, incident response, and network defense.

5. Lab Scenario

A web application hosted on an internal server has been identified as vulnerable to attacks such as SQL injection and cross-site scripting. To mitigate these risks, a Web Application Firewall must be deployed to inspect and filter incoming HTTP traffic.

Students will install ModSecurity on the Ubuntu web server and configure it with the OWASP Core Rule Set. After enabling the WAF, students will repeat the web attacks performed in the previous lab and observe how the firewall detects and blocks malicious requests.

This exercise demonstrates how defensive security mechanisms protect vulnerable applications.

6. Lab Environment Architecture

The lab environment includes an attacker machine and a protected web server.

```
Cybersecurity Lab Network

+---------------------+           +----------------------------+
| Kali Linux VM       |           | Ubuntu Server VM           |
|                     |           |                            |
| Web Browser         | --------> | Apache Web Server          |
| Burp Suite          |           | ModSecurity WAF            |
|                     |           | DVWA Web Application       |
|                     |           | MySQL Database             |
+---------------------+           +----------------------------+
```

The WAF analyzes HTTP requests before they reach the web application.

7. Lab Requirements

Hardware Requirements

Resource	Requirement
RAM	8 GB
Storage	40 GB
CPU	Virtualization enabled

Software Requirements

Software	Purpose
VirtualBox	Virtualization platform
Kali Linux	Attacker machine
Ubuntu Server	Web server
DVWA	Vulnerable web application
Apache	Web server
ModSecurity	Web Application Firewall
OWASP CRS	WAF rule set

8. Step 1: Verify Web Server and DVWA

Ensure that the web server and DVWA application are running.

Check Apache service:

```
sudo systemctl status apache2
```

Open a browser in Kali Linux and verify that DVWA loads:

```
http://192.168.56.20/DVWA
```

Login using:

Field	Value
Username	admin
Password	password

9. Step 2: Install ModSecurity

On the Ubuntu server, install the ModSecurity module.

```
sudo apt install libapache2-mod-security2 -y
```

Enable the module:

```
sudo a2enmod security2
```

Restart Apache:

```
sudo systemctl restart apache2
```

10. Step 3: Enable ModSecurity Rule Engine

Open the ModSecurity configuration file.

```
sudo nano /etc/modsecurity/modsecurity.conf
```

Locate the following line:

```
SecRuleEngine DetectionOnly
```

Change it to:

```
SecRuleEngine On
```

Save the file and restart Apache:

```
sudo systemctl restart apache2
```

This enables active blocking of malicious requests.

11. Step 4: Install OWASP Core Rule Set

Navigate to the ModSecurity directory:

```
cd /etc/modsecurity
```

Clone the OWASP CRS repository:

```
sudo git clone https://github.com/coreruleset/coreruleset.git
```

Copy the configuration template:

```
sudo cp coreruleset/crs-setup.conf.example coreruleset/crs-setup.conf
```

Link the CRS rules to ModSecurity:

```
sudo ln -s /etc/modsecurity/coreruleset/rules /usr/share/modsecurity-crs
```

Restart Apache:

```
sudo systemctl restart apache2
```

12. Step 5: Perform SQL Injection Attack

Open DVWA from Kali Linux.

Navigate to:

```
DVWA → SQL Injection
```

Enter the payload:

```
' OR 1=1 --
```

Submit the request.

If the WAF rule triggers, the browser will display:

```
403 Forbidden
```

This indicates that the attack was blocked.

13. Step 6: Perform Cross-Site Scripting Attack

Navigate to:

DVWA → XSS (Reflected)

Enter the payload:

```
<script>alert('XSS')</script>
```

Submit the request.

If ModSecurity detects the attack, the request will be blocked.

14. Step 7: View WAF Logs

Open the ModSecurity audit log.

```
sudo tail -f /var/log/apache2/modsec_audit.log
```

Example log entry:

```
SQL Injection Attack Detected
Rule ID: 942100
Severity: CRITICAL
```

This log shows the rule triggered by the attack.

15. Step 8: Analyze WAF Detection

Compare attack attempts with WAF responses.

Attack	Detection
SQL Injection	Blocked by CRS SQL injection rule
XSS	Blocked by CRS XSS rule

This demonstrates how the WAF filters malicious HTTP requests.

16. Expected Results

Students should observe:

Result	Description
ModSecurity installed	WAF operational
OWASP CRS enabled	Security rules active
SQL injection attempt blocked	WAF detected malicious request
XSS payload blocked	Script execution prevented
Security logs generated	Attack activity recorded

17. Deliverables

Students must submit screenshots showing:

1. ModSecurity installation confirmation
2. Apache service running with ModSecurity enabled
3. SQL injection attack attempt
4. WAF blocking the request (403 Forbidden)
5. ModSecurity log entry showing detected attack

18. Discussion Questions

1. What is the primary purpose of a Web Application Firewall?
2. How does ModSecurity detect malicious HTTP requests?
3. Why is the OWASP Core Rule Set important for WAF protection?
4. What limitations exist when using WAFs to protect web applications?

19. Conclusion

In this lab, students implemented a Web Application Firewall using ModSecurity to protect a vulnerable web application. By enabling the OWASP Core Rule Set, the firewall was able to detect and block malicious HTTP requests such as SQL injection and cross-site scripting attacks.

Students also analyzed WAF logs to understand how security rules detect and record attack activity. This demonstrates the importance of defensive security mechanisms in protecting web applications from exploitation.

The Web Application Security Lab Series has now guided students through the full lifecycle of web security assessment, from environment setup and vulnerability discovery to exploitation and defensive protection.